

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

Prueba de concepto y plan de trabajo

Versión 3, con matriz de cumplimiento, diagrama de la base de datos y enlace al repositorio

Sistema escalable de seguridad residencial

Hamed Valdez Jason Alejandro

1290897

José Isabel García Rocha

Proyecto de Carrera

5 de octubre de 2026

Repositorio: <https://github.com/HamedJason/ProyectoDeCarrera>

Versión del código descrita: commit f72fce5 (enlace en la sección 13)

1. Objetivo y estado actual

El objetivo de esta etapa es comprobar que la arquitectura seleccionada en el taller anterior puede implementarse realmente, haciendo que un dato proveniente de una entrada física recorra la cadena completa hasta una pantalla en el teléfono, y que el camino inverso permita controlar una salida. Este documento es la segunda versión del taller. La primera se entregó cuando solo existía el firmware provisional con un push button, el backend y la base de datos. Desde entonces el prototipo avanzó de forma importante y esta versión describe lo que existe y se probó, separando con cuidado lo que ya se verificó de lo que todavía está pendiente.

La cadena implementada es la siguiente. Los sensores magnéticos MC-38 y el sensor de movimiento AM312 se conectan a un nodo inalámbrico ESP32-C3. El nodo envía cada evento mediante ESP-NOW a un ESP32 que funciona como hub de la vivienda. El hub aplica la lógica de armado y alarma, controla la salida audible y envía los eventos por Wi-Fi al backend, que está desplegado en un servidor privado virtual con contenedores. El backend los valida, los guarda en PostgreSQL y los sirve a una aplicación web instalable en el teléfono, que además recibe notificaciones push cuando suena la alarma.

Lo que cambió respecto a la primera versión. El frontend, que estaba pendiente, ya existe y consume el backend desplegado. Los sensores definitivos (MC-38 y AM312) sustituyeron al push button. La comunicación entre nodos y hub, que estaba programada para la semana 2 del cronograma, ya se implementó con ESP-NOW. El modelo de datos dejó de ser una sola tabla de mediciones y ahora registra sensores de distintos tipos, y la aplicación permite agregar, renombrar y eliminar puertas, ventanas y detectores de movimiento. También se adelantaron el armado y desarmado, el modo silencioso, el registro de acciones y las notificaciones push, que estaban programados para semanas posteriores.

Lo que sigue pendiente. La cámara, la sirena real con su etapa de potencia, los retardos de entrada y salida, la migración a MQTT y las pruebas cuantitativas de los indicadores. La única serie cuantitativa disponible sigue siendo la de diez transmisiones del 16 de septiembre, que se conserva en la sección 12 porque es la única medición formal. Las pruebas del enlace ESP-NOW, de las notificaciones y de la recuperación ante pérdida de red se hicieron de forma funcional, sin series medidas, y así se indica en cada caso.

Cambios de la versión 3. Esta versión no modifica el prototipo descrito, que corresponde al código del commit f72fce5 del repositorio. Reorganiza la entrega para que cada criterio de la actividad se pueda verificar sin buscar: agrega una matriz de cumplimiento que indica, para cada criterio, si se cumple y en qué sección está la evidencia (sección 18); incorpora el diagrama entidad-relación de la base de datos y la comprobación de la base de datos mínima (secciones 7.1 y 7.2); documenta el procedimiento automatizado para la medición del tiempo hasta el frontend y las pruebas de rechazo (sección 12.6); y añade el enlace al repositorio en la portada y en las evidencias. Actualiza además el presupuesto con un rollo de filamento para impresión 3D (\$500.00) y compara la cámara y el costo total con lo previsto en el Taller 4 (sección 2).

2. Alcance de esta etapa y decisiones tomadas

Cada decisión de sustitución temporal se documenta con su justificación, porque ninguna modifica la arquitectura aprobada en el taller anterior. La tabla muestra la situación actual y marca en negritas lo que cambió desde la primera versión.

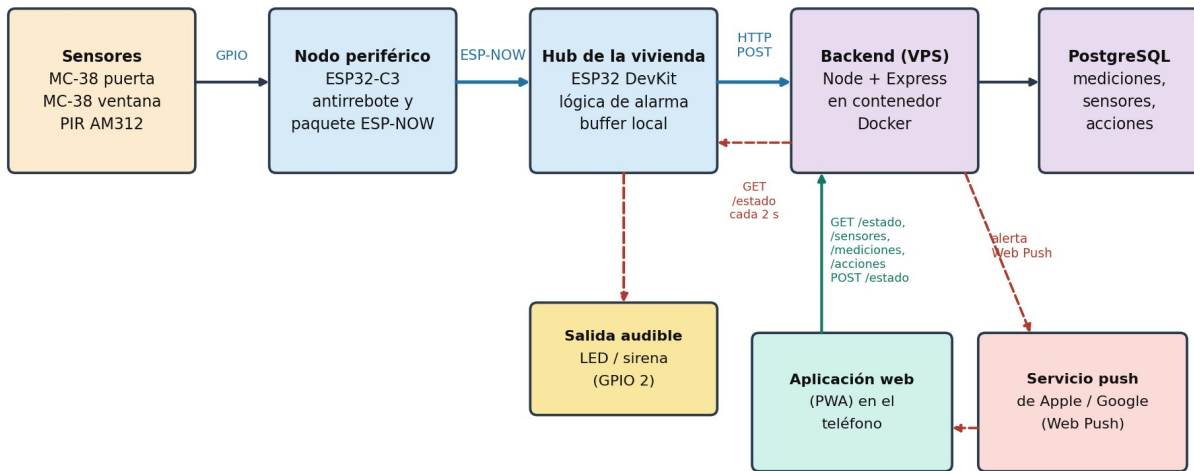
Elemento	Solución actual	Justificación técnica
Enlace nodo a hub	ESP-NOW entre un ESP32-C3 y el ESP32 hub	Es la tecnología definida en el Taller 4. Los sensores no necesitan conocer el Wi-Fi de la vivienda. El nodo busca por sí solo el canal del hub y solo requiere conocer su dirección MAC
Transporte hub a backend	HTTP con POST y GET hacia un puerto alterno del servidor	MQTT con TLS sigue siendo la decisión del Taller 4 y está programado para la semana 6. Los puertos 80 y 443 del servidor están ocupados por otro proyecto, por lo que el firmware usa un puerto alterno y, por ahora, HTTP
Entrada física	Contactos MC-38 y sensor PIR AM312 definitivos	Ya se recibió el material. Ambos tipos de contacto seco se leen con resistencia de elevación interna y el PIR con entrada simple. El firmware admite varios contactos por nodo mediante una tabla
Salida física	LED con resistencia de 220 ohm en el GPIO 2 del hub	La bocina de 12 V requiere relevador o MOSFET y una fuente independiente, que todavía no se adquieren. El LED recorre exactamente el mismo camino de control
Entrega del comando al hub	Consulta periódica de GET /estado cada 2 segundos	Con HTTP no existe un canal persistente hacia el dispositivo. Se sustituye por MQTT en la semana 6
Aplicación del usuario	Aplicación web instalable (PWA) servida por el propio backend	Una sola base de código funciona en Android y en iOS, no depende de una tienda de aplicaciones y permite notificaciones push. Es una decisión propia que conviene confirmar con el docente, porque el alcance menciona una aplicación móvil Android e iOS (sección 17)
Notificaciones	Web Push propio con claves VAPID, sin servicio de terceros	El servidor envía la alerta al servicio de push de Apple o de Google. Evita depender de una suscripción externa, en línea con el aporte planteado en los talleres anteriores
Marca de tiempo	Asignada por el backend	El ESP32 no cuenta con reloj de tiempo real. El firmware envía además un número de registro consecutivo
Módulo de cámara	ESP32-S3 CAM con sensor OV5640, en lugar de la ESP32-CAM con OV2640 considerada en el Taller 4	El módulo con OV5640 se compró en el pedido (sección 15.2). Es un cambio de componente y no de arquitectura: la cámara sigue siendo un dispositivo independiente del hub y enviará las fotografías por HTTPS en un canal separado del tráfico de sensores. Todavía no está integrada
Presupuesto	\$3120.24 pesos en material, sin considerar envíos	El Taller 4 estimaba \$2040 para un hub, tres nodos, cámara y sirena. El pedido real incluyó más piezas (8 nodos ESP32-C3 en lugar de 3, kits de componentes y de soldadura) y se agregó un rollo de filamento para impresión 3D. El detalle está en la sección 15

3. Flujo implementado

El diagrama muestra la cadena tal como está codificada. La aplicación no consulta la base de datos de forma directa, sino que solicita la información al backend, de modo que la base nunca queda expuesta. El hub es el único dispositivo de la vivienda que se conecta a Internet. Los nodos solo hablan con él por ESP-NOW.

Flujo implementado del prototipo

Cada bloque indica la tecnología utilizada



Azul: escritura de eventos. Verde: lectura desde la aplicación. Rojo discontinuo: control de la salida y notificación.

Figura 1. Flujo implementado. En azul el camino de escritura, en verde el de lectura y en rojo el control de la salida y la notificación.

4. Diagrama de conexión

El montaje usa dos placas. El nodo ESP32-C3 lleva los dos contactos magnéticos y el sensor de movimiento. El hub ESP32 DevKit lleva la salida de alarma. Los contactos son contactos secos que aprovechan la resistencia de elevación interna, por lo que no requieren resistencias externas. El AM312 se alimenta con 3.3 V y su salida se conecta a una entrada simple, sin resistencia interna, porque su salida entrega muy poca corriente.

Diagrama de conexión del prototipo

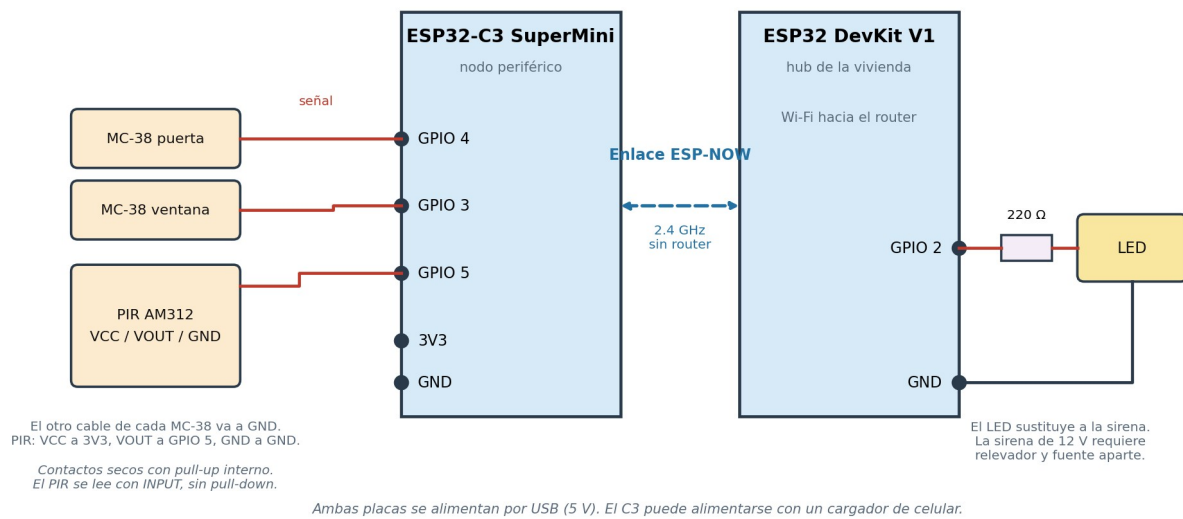


Figura 2. Conexión del nodo ESP32-C3 con sus sensores y del hub ESP32 con la salida. Pines por omisión del firmware.

Sensor o salida	Pin del ESP32-C3 / hub	Observación
MC-38 de puerta (sensor - puerta-01)	GPIO 4 del C3	Contra GND. Abierto se lee como nivel alto
MC-38 de ventana (sensor - ventana-01)	GPIO 3 del C3	Se agrega con una línea en la tabla de contactos del firmware
PIR AM312 (pir-sala-01)	GPIO 5 del C3	Estabilización de 45 s al arrancar
Salida audible (LED o relevador)	GPIO 2 del hub	Intermitente durante 30 s al sonar la alarma

5. Sistema embebido

Hay dos programas, ambos escritos para el entorno de Arduino. El del nodo ESP32-C3 lee los sensores y los transmite. El del hub recibe, decide y reenvía. Los dos comparten la definición del paquete ESP-NOW y están en el repositorio.

5.1 Nodo periférico ESP32-C3

El nodo realiza cuatro tareas: leer los contactos, leer el PIR, encontrar al hub y transmitir los eventos con confirmación.

- **Contactos magnéticos.** La lectura no se envía directamente. Un contacto mecánico produce rebotes y cada accionamiento generaría varios eventos falsos. El nodo confirma un cambio solo cuando se mantiene estable durante 50 ms. Cada contacto tiene su propio filtro, y se pueden declarar hasta seis en una tabla con identificador, zona y pin.
- **Sensor de movimiento.** El AM312 necesita entre 30 y 60 segundos para estabilizarse, por lo que el firmware ignora sus pulsos durante 45 s. Después detecta el flanco de subida por interrupción y,

como respaldo, por sondeo. Tras un evento bloquea 8 segundos los siguientes para no inundar el sistema mientras la persona sigue en movimiento.

- **Búsqueda del hub.** Para que dos radios ESP-NOW se oigan deben estar en el mismo canal de Wi-Fi, y el hub queda en el del router. El nodo prueba los canales del 1 al 13 enviando un paquete y observando si se confirma la entrega. Se queda en el primero que responde y vuelve a buscar si tres envíos consecutivos fallan. Por eso basta copiar una sola vez la dirección MAC del hub en el firmware del nodo, y cambiar de red no exige tocar el nodo.
- **Confiabilidad.** Cada paquete lleva un número de secuencia, el identificador del nodo y del sensor, la variable, la zona y el valor. Los envíos se reintentan hasta tres veces y, si el hub no está disponible, se guardan en una cola de 16 eventos. Además cada 30 segundos el nodo manda el estado actual de cada contacto, lo que registra sensores nuevos y corrige eventos perdidos.

5.2 Hub ESP32

El hub recibe los paquetes en una cola, descarta duplicados por número de secuencia y aplica la lógica de la vivienda. Consulta cada 2 segundos al backend para saber si el sistema está armado o en modo silencioso. Si llega una apertura o un movimiento con el sistema armado, enciende la salida de forma intermitente durante 30 segundos y avisa al backend para que registre la alarma y envíe la notificación. Si el modo silencioso está activo, registra y notifica sin activar la salida. Con el sistema desarmado solo registra el evento.

Todos los eventos se envían con `POST /mediciones` y el hub mide el tiempo entre el envío y la respuesta. Si no hay confirmación, reintenta hasta tres veces y luego guarda el evento en un buffer local de veinte registros para sincronizarlo al recuperar la conexión. La reconexión a la red no detiene el programa, de modo que el hub sigue atendiendo eventos aun sin Internet. La primera vez que el hub ve un sensor lo registra sin disparar alarma, para que un reinicio no genere alarmas falsas.

5.3 Identificación de cada medición

Cada evento recibe un número de registro consecutivo generado por el hub. Esto permite detectar si alguna medición se perdió en el trayecto, algo que una marca de tiempo por sí sola no revelaría. La fecha y hora definitivas las asigna el backend porque el ESP32 no cuenta con reloj de tiempo real.

6. Backend

El backend está desarrollado con Node.js y Express y se despliega en un servidor privado virtual propio con contenedores: uno para la API y otro para PostgreSQL. La base de datos no se expone a Internet y solo es alcanzable desde la red interna del servidor. La misma API sirve la aplicación web, por lo que la app y el backend comparten origen. Un solo comando levanta todo el despliegue y las tablas se crean solas al arrancar.

Se eligió un servidor propio porque los planes gratuitos suspenden la aplicación tras unos minutos sin tráfico, lo que distorsionaría la medición de los tiempos de registro remoto, y porque es coherente con el aporte de contar con un backend propio sin suscripciones obligatorias.

Endpoints implementados

Endpoint	Función
POST /mediciones	Registra un evento del hub. Exige el token del dispositivo y valida los campos. Si el sensor es nuevo, lo da de alta automáticamente
GET /mediciones, GET /mediciones/ultima	Historial de eventos de una vivienda y dato más reciente
GET /estado, POST /estado	Consulta y cambio del estado de armado y del modo silencioso. El hub lo consulta cada 2 s y la aplicación lo modifica
POST /alarma	El hub informa una alarma con el sensor que la originó. El backend la registra y envía la notificación
GET /sensores, POST /sensores, PATCH /sensores/:id, DELETE /sensores/:id	Registro de sensores de cada vivienda. Permite nombrar cada sensor y elegir si un contacto es puerta o ventana
GET /acciones	Trazabilidad de quién hizo qué y cuándo (armar, desarmar, silenciar, eliminar sensor)
GET /comando, POST /comando	Control manual de la salida, usado para probar la sirena sin un evento real
GET /push/clave, POST /push/suscribir, POST /push/cancelar, POST /push/prueba	Suscripción de dispositivos a las notificaciones push y envío de una notificación de prueba
GET /salud, GET /configuracion	Comprobación del servicio y de la base de datos, y funciones habilitadas en este servidor

Validación y respuesta ante un dato incorrecto

La validación separa los campos ausentes de los de tipo incorrecto y enumera todos los problemas en lugar de devolver un error genérico, lo que facilita corregir el firmware sin revisar los registros del servidor. Ante una medición válida responde con código 201 y el registro creado. Ante una incorrecta responde con código 400 y no registra nada:

```
{
  "ok": false,
  "mensaje": "La medicion no pudo registrarse porque el dato es incorrecto.",
  "errores": [
    "Falta el campo obligatorio \"sensor_id\".",
    "El campo \"valor\" debe ser numerico."
  ]
}
```

El hub debe incluir el encabezado X-Device-Token. No es el esquema de seguridad definitivo, que se define en los talleres correspondientes, sino un control mínimo para que una petición anónima no escriba en la base. Conviene señalar una limitación conocida: mientras la comunicación del hub sea HTTP, el token viaja sin cifrar, y el token sigue escrito en el firmware. Ambas cuestiones están en la matriz de riesgos y se corrigen antes del prototipo alfa.

7. Base de datos

La base de datos creció de una tabla a seis. Se mantuvo la tabla de mediciones de la primera versión, porque conserva los eventos tal como llegan, y se agregaron las tablas necesarias para representar

sensores, estado de la vivienda, acciones del usuario y suscripciones push. Los campos `vivienda_id`, `zona`, `nodo_id` y `sensor_id` siguen siendo texto para conservar la jerarquía del proyecto sin adelantar el modelo relacional definitivo, que se construye en la semana 4. Las tablas se crean automáticamente al arrancar el servicio, así que volver a levantar el backend no exige pasos manuales.

Tabla	Contenido	Campos principales
mediciones	Cada evento recibido de un sensor	id, vivienda_id, zona, nodo_id, sensor_id, variable, valor, unidad, numero_registro, creado_en
sensores	Registro de puertas, ventanas y detectores de movimiento de cada vivienda	id, vivienda_id, sensor_id, nodo_id, tipo (puerta, ventana o movimiento), nombre, zona, creado_en. Clave única por vivienda y sensor
estado_vivienda	Estado persistente de cada vivienda	vivienda_id, armado, modo_silencioso, actuador_activo, alarma_activa, alarma_motivo, alarma_sensor, actualizado_en
acciones	Trazabilidad de las acciones del usuario	id, vivienda_id, accion, valor_anterior, valor_nuevo, usuario, creado_en
suscripciones_push	Dispositivos que aceptaron recibir notificaciones	id, vivienda_id, endpoint, p256dh, auth, agente, creado_en
ajustes_servidor	Claves VAPID del servidor, generadas una sola vez	clave, valor

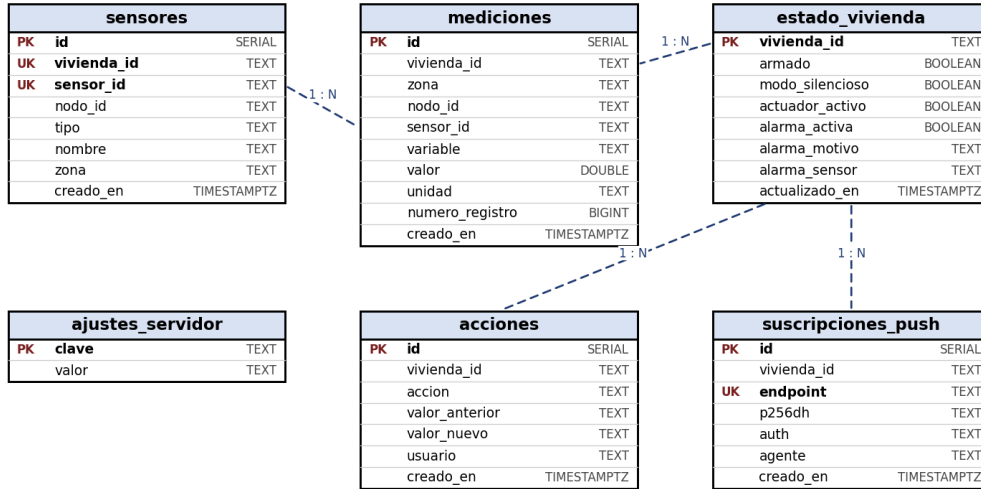
Que el registro de sensores sea una tabla y no parte del firmware es lo que permite el requisito de escalabilidad: agregar una puerta, una ventana o un detector no exige modificar el esquema ni el código del backend. Un sensor aparece en la tabla la primera vez que reporta un dato, o cuando el usuario lo da de alta en la aplicación. Si el usuario elimina un sensor, el backend recuerda la baja para no volver a registrarlo cuando el nodo lo siga reportando. La consulta principal de la aplicación es la siguiente:

```
SELECT id, vivienda_id, zona, nodo_id, sensor_id, variable,
       valor, unidad, numero_registro, creado_en
FROM mediciones
WHERE vivienda_id = 'casa-001'
ORDER BY creado_en DESC
LIMIT 50;
```

7.1 Diagrama entidad-relación

El diagrama muestra las seis tablas del esquema en el commit f72fce5, con sus claves y los tipos de cada campo. Las relaciones son lógicas: la vivienda se identifica con el texto `vivienda_id` y cada sensor con el par `vivienda_id` y `sensor_id`. No se declararon llaves foráneas para no adelantar el modelo relacional definitivo, que se construye en la semana 4 del cronograma. El archivo `schema.sql` del repositorio contiene las sentencias completas.

Estructura de la base de datos (PostgreSQL, commit b5be6ba)



PK: clave primaria UK: clave única (en sensores, el par vivienda_id + sensor_id). Línea punteada: relación lógica por vivienda_id y sensor_id. Se conservan como texto, sin llaves foráneas, para no adelantar el modelo relacional definitivo (semana 4 del cronograma).

Figura 3. Estructura de la base de datos. Las líneas punteadas son relaciones lógicas por vivienda_id y sensor_id.

7.2 Comprobación de la base de datos mínima

La tabla relaciona cada criterio de la base de datos mínima con el elemento del esquema que lo cumple.

Criterio de la base de datos mínima	Cómo se cumple	Dónde verificarlo
Una tabla o colección	La tabla mediciones recibe todos los eventos de los sensores. El resto de las tablas es complementario	Tabla de la sección 7 y diagrama 7.1
Identificador del registro	Campo id de tipo SERIAL, clave primaria. Lo asigna la base de datos y se devuelve al hub en la respuesta 201	Diagrama 7.1. Columna Id de la serie en 12.1
Variable y valor	Campos variable (estado_puerta o movimiento) y valor (numérico). El estado abierto se guarda como 1 y cerrado como 0	Diagrama 7.1. Serie de 12.1
Unidad, cuando corresponda	Campo unidad. Los eventos de contacto y movimiento se guardan con la unidad estado. Es opcional para otras variables	Diagrama 7.1
Fecha y hora	Campo creado_en de tipo TIMESTAMPTZ con valor por omisión NOW(). La asigna el backend porque el ESP32 no tiene reloj de tiempo real. El número de registro del hub (campo numero_registro) permite detectar eventos perdidos	Sección 5.3 y diagrama 7.1
Mínimo diez registros reales	Los identificadores 6 al 15 de la serie del 16 de septiembre son diez registros reales enviados por el hub, a los que se suman los generados por el uso posterior del prototipo con el nodo C3 y los sensores	Tabla de la sección 12.1

Criterio de la base de datos mínima	Cómo se cumple	Dónde verificarlo
Una consulta para recuperar la información	Consulta de las últimas 50 mediciones de una vivienda, ordenadas por fecha, respaldada por un índice sobre vivienda y fecha. El frontend la usa a través de GET /mediciones	Consulta de la sección 7 y esquema schema.sql

8. Frontend

La aplicación es una página web sin framework, servida por el propio backend. Se puede agregar a la pantalla de inicio de Android y de iOS y se comporta como una aplicación independiente. Consume únicamente la API del backend. Sus funciones actuales son:

- **Estado y armado.** Muestra si el sistema está armado y permite armarlo, desarmarlo y activar el modo silencioso.
- **Tarjetas por sensor.** Cada puerta, ventana y detector de movimiento aparece como una tarjeta con su nombre, su zona y su estado, con una animación que refleja lo que ocurre (puerta abierta, movimiento detectado, alarma). Al tocar una tarjeta se puede cambiar el nombre, la zona, o elegir si un contacto es puerta o ventana, y también eliminarlo.
- **Sirena.** Una tarjeta propia permite probar la salida audible sin generar un evento real.
- **Historial.** Línea de tiempo con los eventos y las acciones del usuario, donde cada entrada nombra el sensor que la originó.
- **Avisos.** Interruptor para activar las notificaciones push en el dispositivo, con un botón para enviar una notificación de prueba.

Estados de la interfaz

La actividad solicita mensajes básicos de carga, ausencia de datos y error de comunicación. Se tratan como estados distintos porque significan cosas diferentes para quien usa la aplicación:

- Durante la primera consulta se muestra un indicador de carga.
- Si la consulta responde pero no hay registros, se indica que todavía no hay mediciones.
- Si la comunicación falla, se muestra el motivo y un botón para reintentar, distinguiendo el tiempo de espera agotado del resto de los errores.

La aplicación actualiza los datos periódicamente y cuando el usuario vuelve a ella tras dejarla en segundo plano. Los elementos se actualizan en su lugar, sin reconstruir la pantalla, para que las animaciones no se reinicien.

9. Control de la salida y armado

El armado y desarmado remoto es una función central, y por eso el camino de control ya funciona de extremo a extremo. El usuario cambia el estado en la aplicación, el backend lo guarda en la base de datos y lo registra en la tabla de acciones, y el hub lo consulta cada dos segundos y lo aplica sobre la salida. Como el estado vive en la base de datos, sobrevive a un reinicio del backend y el hub lo recupera al reiniciarse.

La consulta periódica se utiliza porque con HTTP no existe un canal permanente hacia el dispositivo. Es adecuada para la demostración, pero introduce un retraso de hasta dos segundos y genera tráfico constante, por lo que se sustituye por MQTT en la semana 6. Los retardos de entrada y de salida, que

permiten al residente entrar y desarmar sin disparar la alarma, todavía no están implementados y se abordan en la semana 5.

10. Notificaciones push

Las notificaciones se envían con Web Push, el estándar con el que el navegador recibe avisos aunque la aplicación esté cerrada. El backend genera una sola vez un par de claves VAPID que lo identifican ante los servicios de push de Apple y de Google, y las conserva en la base de datos. Cuando el usuario activa los avisos, el navegador crea una suscripción que se guarda en el servidor. Al registrarse una alarma, el backend envía el mensaje a cada suscripción y elimina las que el servicio ya no reconoce. El envío no bloquea la alarma: si el servicio de push tarda o falla, el registro del evento y la sirena no se retrasan.

En iPhone las notificaciones exigen iOS 16.4 o superior, abrir la aplicación por HTTPS y haberla agregado a la pantalla de inicio. La aplicación detecta esos casos y explica al usuario qué falta. También se dejó opcional el envío por ntfy, que queda desactivado si no se configura.

Resultado de las primeras pruebas. Las notificaciones llegan, pero se observó un retraso mayor al esperado. Todavía no se midió. Para el indicador de notificación del Taller 6 (30 segundos o menos) se requiere una prueba cronometrada de diez eventos, que aparece en el plan de la sección 12.3. Entre las causas posibles están la entrega diferida del servicio de push del teléfono, el modo de ahorro de energía del dispositivo y el tiempo que tarda el hub en confirmar y reportar la alarma. La medición separará esos tramos.

Las notificaciones push exigen que la aplicación se abra por HTTPS. Para cumplirlo, la aplicación se publica a través del servidor web que ya gestiona los certificados del otro proyecto alojado en el servidor, con un nombre de dominio propio de este proyecto. El firmware del hub sigue usando HTTP hacia el puerto alterno, como se indica en la sección 2.

11. Estado de verificación a la fecha

Esta sección distingue lo comprobado de lo pendiente, para no presentar como resultado algo que todavía no se ha medido.

Componente o actividad	Estado	Observación
Firmware del nodo ESP32-C3	Funcionando	Lee los contactos y el PIR y envía los eventos por ESP-NOW. Admite varios contactos mediante una tabla
Firmware del hub ESP32	Funcionando	Recibe los eventos, aplica la lógica de armado, controla la salida y envía al backend con reintentos y buffer
Enlace ESP-NOW nodo a hub	Funcionando en banco	El hub recibe y registra los eventos del nodo. El alcance, la pérdida de paquetes y la latencia todavía no se miden
Sensor PIR AM312	Funcionando	Tras corregir el modo del pin (sección 12.5). Falta medir detección y falsos positivos
Backend y endpoints	Desplegado y funcionando	Contenedores en el servidor, accesible desde Internet. Incluye el registro de sensores y las notificaciones push
Base de datos	Funcionando	PostgreSQL con seis tablas. Se crea sola al arrancar
Frontend	Funcionando	Aplicación web con tarjetas por sensor, armado, modo silencioso, historial y avisos

Componente o actividad	Estado	Observación
Notificaciones push	Funcionando, con retraso	Llegan al dispositivo. El tiempo de entrega no se ha medido
Montaje físico	Provisional	Sensores definitivos sobre protoboard. La salida es un LED en lugar de la sirena
Serie de diez transmisiones	Realizada el 16 de septiembre	Con el hub enviando directamente al backend. Sección 12.1
Prueba de dato válido e incorrecto	Realizada	Códigos 201 y 400 confirmados
Prueba de token ausente (401)	Pendiente	El backend ya lo rechaza. Falta ejecutar la serie formal
Prueba de pérdida de conexión	Observada, pendiente formalizar	El hub reintentó tres veces y conservó los eventos en el buffer. Falta la prueba planificada con conteo
Cámara, sirena real, retardos de entrada y salida	Pendiente	Semanas 3 y 5 del cronograma
Comunicación MQTT sobre ESP32	Probada de forma preliminar	Prueba independiente previa. No está integrada al prototipo

12. Pruebas realizadas

12.1 Serie inicial de transmisiones (16 de septiembre de 2026)

La primera serie se ejecutó con el nodo conectado a una red doméstica de 2.4 GHz y el backend en el servidor privado virtual. En ese momento el ESP32 leía un push button y enviaba directamente al backend, sin nodo intermedio. Se conserva porque es la medición formal de la cadena hub a backend. El firmware mide el tiempo entre el envío de la petición y la recepción de la respuesta, que corresponde al trayecto de la red local, el enlace a Internet, el procesamiento y la escritura en la base de datos. No equivale todavía al tiempo completo de sensor a pantalla.

No.	Evento generado	Valor enviado	Código HTTP	Latencia (ms)	Id asignado en la base
1	Apertura	1	201	398	6
2	Cierre	0	201	305	7
3	Apertura	1	201	394	8
4	Cierre	0	201	408	9
5	Apertura	1	201	309	10
6	Cierre	0	201	338	11
7	Apertura	1	201	360	12
8	Cierre	0	201	345	13
9	Apertura	1	201	349	14
10	Cierre	0	201	300	15

Indicador de la prueba	Resultado obtenido	Observación
Transmisiones realizadas	10	Definido por la actividad
Transmisiones exitosas, código 201	10	Tasa de éxito del 100 por ciento
Transmisiones fallidas	0	No se requirió ningún reintento
Latencia promedio del nodo al backend	350.6 ms	Medida por el firmware en cada envío
Latencia mínima observada	300 ms	Transmisión número 10
Latencia máxima observada	408 ms	Transmisión número 4
Desviación estándar de la latencia	39.5 ms	Dispersión baja entre transmisiones
Registros almacenados en la base de datos	15	Identificadores 6 al 15 en esta serie, más los previos de las pruebas manuales

La latencia promedio fue de 350.6 ms, con mínimo de 300 y máximo de 408. Estos tiempos están muy por debajo del indicador de registro remoto del proyecto, que fija un máximo de diez segundos en al menos el noventa por ciento de las pruebas con conectividad estable. Esta serie no se repitió con la arquitectura actual. Con el nodo ESP-NOW y el hub en la cadena, la latencia de sensor a base de datos debe volver a medirse, y está incluida en el plan de la sección 12.3.

12.2 Pruebas funcionales de la segunda etapa

Estas pruebas verifican que cada función opere, pero no son series medidas. Se anota lo observado sin atribuirle cifras que no se obtuvieron.

Prueba	Comportamiento esperado	Resultado observado
El nodo C3 envía eventos de contacto y de movimiento al hub	El hub imprime cada evento recibido con el nodo, el sensor y el valor	Confirmado. El monitor serie del hub muestra líneas del tipo [ESP-NOW] evento de nodo-c3-01: pir-sala-01 valor=1
El nodo encuentra al hub sin configurar el canal	Tras escanear los canales, el nodo se enlaza y empieza a enviar	Confirmado. Solo se copió la dirección MAC del hub en el firmware del nodo
Un sensor nuevo aparece en la aplicación sin alta manual	Tras su primer dato, aparece una tarjeta que se puede renombrar y cambiar de tipo	Confirmado con el sensor de movimiento y el contacto de puerta
El hub sin acceso al servidor conserva los eventos	Tras tres reintentos, el evento pasa al buffer local	Observado. Con el servidor inalcanzable (por una dirección mal configurada), el monitor mostró Reintento 2, Reintento 3 y Eventos pendientes: 6. Falta verificar la sincronización posterior con conteo
Armado y modo silencioso desde la aplicación	El estado cambia en el hub y se registra la acción	Implementado y usado durante las pruebas. Falta medir el tiempo de respuesta
Notificación push al sonar la alarma	El dispositivo recibe el aviso aunque la aplicación esté cerrada	Las notificaciones llegan, con un retraso mayor al esperado. Sin medir

12.3 Plan de pruebas cuantitativas pendientes

Las siguientes series corresponden a indicadores comprometidos del proyecto y todavía no se han ejecutado. Se dejan definidas con su procedimiento y sus columnas de resultado vacías, para no presentar cifras estimadas como medidas.

Serie	Procedimiento	Cantidad	Indicador que alimenta	Resultado
Detección de apertura	Abrir y cerrar cada contacto y comparar con los registros de la base de datos	100 accionamientos	Detección mayor o igual a 95 %	_____
Detección de movimiento y falsos positivos	Pasar frente al PIR a 3 m y dejar la zona vacía un tiempo fijo	50 pasos y 30 min sin presencia	Detección y falsos positivos menores o iguales a 5 %	_____
Alcance y pérdida de ESP-NOW	Enviar paquetes del nodo al hub a 1 m, 5 m, con una pared y con varias paredes	100 paquetes por condición	Pérdida, reintentos y alcance	_____
Latencia de sensor a base de datos	Medir desde el accionamiento hasta el registro con el nodo y el hub en la cadena	20 eventos	Registro remoto en 10 s o menos	_____
Tiempo de alarma local	Medir del evento recibido por el hub al cambio de la salida	20 eventos	Alarma local en 1 s o menos	_____
Notificación push	Cronometrar del evento a la llegada del aviso al teléfono	10 eventos	Notificación en 30 s o menos	_____

Serie	Procedimiento	Cantidad	Indicador que alimenta	Resultado
Pérdida de conexión	Desconectar el Wi-Fi del hub, generar 5 eventos, reconectar y contar los registros	5 eventos	100 % conservados y sin duplicados	_____
Alta de sensores y zonas	Dar de alta 3 sensores o zonas adicionales desde la aplicación y revisar que el repositorio no cambie	3 altas	Escalabilidad sin rediseño	_____

12.4 Pruebas de dato válido e incorrecto

Las dos primeras pruebas se ejecutaron contra el backend desplegado. El backend no se limita a rechazar el dato incorrecto, sino que enumera por separado cada problema encontrado: en la prueba devolvió a la vez que faltaba `sensor_id` y que `valor` no era numérico. Las tres restantes siguen pendientes.

Prueba	Comportamiento esperado	Comportamiento observado
Envío de una medición completa y válida	El backend responde código 201 y devuelve el registro creado con su identificador y su marca de tiempo	Confirmado. Respondió código 201 con el mensaje de registro correcto y devolvió el objeto creado con su identificador y la fecha asignada por el servidor
Envío de una medición sin el campo <code>sensor_id</code> y con valor no numérico	El backend responde código 400, enumera los problemas encontrados y no registra el dato	Confirmado. Respondió código 400 con los dos errores enumerados por separado: falta el campo obligatorio <code>sensor_id</code> , y el campo <code>valor</code> debe ser numérico. No se generó ningún registro
Envío sin el encabezado del token del dispositivo	El backend responde código 401 y no registra la medición	_____
Pérdida temporal de conexión en el nodo	El nodo reintenta hasta tres veces, guarda el evento en el buffer local y lo sincroniza al recuperar la red	_____
Backend no disponible durante la consulta del frontend	La aplicación muestra un mensaje de error de comunicación y ofrece reintentar, sin cerrarse	_____

12.5 Registro de problemas y correcciones

Durante la puesta en marcha se presentaron los siguientes problemas. Los primeros seis corresponden a la primera versión y los últimos cinco a esta etapa. Todos están resueltos.

Problema encontrado	Causa identificada	Corrección aplicada
El servidor web del despliegue no arrancaba, con el mensaje de que el puerto ya estaba asignado	Los puertos 80 y 443 del servidor ya estaban ocupados por otro proyecto alojado en la misma máquina	Se retiró el servidor web del despliegue y el backend se publicó directamente en un puerto alternativo. Como consecuencia se opera sobre HTTP, ya que la emisión del certificado requiere el puerto 80
El firmware no compilaba por no encontrar la librería de manejo de JSON	La librería no estaba instalada en el entorno de desarrollo	Se instaló <code>ArduinoJson</code> en su versión 7. La versión 6 no es compatible con la sintaxis utilizada en el firmware

Problema encontrado	Causa identificada	Corrección aplicada
Las pruebas con peticiones desde la terminal de Windows devolvían un error de llaves sin cerrar	El intérprete de comandos de Windows elimina las comillas del cuerpo JSON antes de entregarlo al programa	Las peticiones de prueba se ejecutan desde la sesión remota en el servidor, donde las comillas simples preservan el contenido. La evidencia es equivalente, ya que lo que se prueba es el backend
El monitor serie mostraba caracteres ilegibles después de cargar el firmware	El puerto quedaba desincronizado al terminar la carga, aun con la velocidad correcta configurada	Se cierra y se vuelve a abrir el monitor, y se reinicia la placa con el botón correspondiente para observar el arranque desde el inicio
Las peticiones iniciales desde la computadora devolvían una página de error en lugar de la respuesta del backend	La dirección utilizada no incluía el puerto, por lo que la petición llegaba al otro proyecto alojado en el mismo servidor	Se corrigió la dirección incluyendo un puerto alternativo en el firmware y en los comandos de prueba
Al desconectar la red, el nodo dejó de responder a la entrada física. Las pulsaciones no generaban ningún evento	La rutina de reconexión esperaba la red dentro de un ciclo que detenía el programa hasta quince segundos. Durante esa espera no se leía el sensor, por lo que las pulsaciones se perdían	Se reescribió la atención de la red para que no detenga el programa: ahora solo consulta el estado y espacia los reintentos, mientras el ciclo principal sigue leyendo el sensor. Sin conexión, el evento pasa directamente al buffer local sin intentar reenvíos
El sensor PIR no detectaba movimiento. Con el multímetro se medía un voltaje constante de alrededor de 1.6 V en su salida	El firmware cargado en la placa era una copia anterior que configuraba el pin con resistencia de bajada interna. La salida del AM312 es débil y esa resistencia la dejaba en un nivel intermedio que nunca llegaba a alto ni a bajo. Al desconectar el cable de la salida, el pin funcionaba, lo que lo delató	El pin se configura como entrada simple, sin resistencia interna, y se agregó una detección de respaldo por sondeo. Se comprobó que la salida volvió a cambiar entre alto y bajo
Error de compilación en el hub: un tipo de dato del protocolo no se reconocía	El entorno de Arduino genera automáticamente los prototipos de las funciones antes de la primera función del archivo, por lo que los tipos usados en sus firmas deben estar definidos antes	Las definiciones de los tipos del protocolo se movieron al inicio, antes de cualquier función
El monitor serie mostraba la dirección MAC 00:00:00:00:00:00	La función usada para leerla devuelve ceros si se consulta antes de que el Wi-Fi esté inicializado	Se lee directamente del chip con la función de bajo nivel, que siempre devuelve la dirección real
La función que recibe los paquetes ESP-NOW no compilaba en algunas versiones del entorno	La firma de la función cambió entre las versiones 4 y 5 del marco de Espressif, y lo mismo ocurre con la de confirmación de envío	Se escribieron ambas variantes y se elige la correcta según la versión con las macros del propio marco
Los envíos al backend fallaban con código -1 y 5002 ms de latencia, y los eventos quedaban en el buffer	La dirección del servidor configurada en el firmware no correspondía al servidor desplegado, por lo que la conexión agotaba el tiempo de espera	Se corrigió la dirección y el puerto. Los eventos se conservaron en el buffer mientras tanto

Dos de estos hallazgos merecen una observación aparte. El primero, de la primera versión, es que el nodo dejaba de atender el sensor justamente cuando la red faltaba, porque la rutina de reconexión bloqueaba el ciclo principal. El error no estaba en el almacenamiento de eventos sino en la espera de la red, y se habría incumplido el indicador de recuperación de conectividad pese a que el buffer estaba bien implementado. El segundo, de esta etapa, es el del PIR: la medición con multímetro permitió separar un problema de hardware de uno de software, y mostró que la causa estaba en una copia desactualizada del

firmware y no en el sensor. En ambos casos probar el comportamiento ante fallas y medir directamente fue más informativo que probar solo el funcionamiento normal.

12.6 Medición del tiempo hasta el frontend y pruebas de rechazo

Para cerrar las pruebas mínimas se incluyó en la rama principal del repositorio, como complemento posterior al código descrito, el script `pruebas/medir_tiempos.js`, que se ejecuta contra el servidor desplegado sin necesitar el hardware. Realiza diez transmisiones consecutivas de un contacto con el mismo formato que usa el hub y, por cada una, registra el código HTTP, la latencia de la respuesta, el identificador asignado en la base de datos y el tiempo que tarda el dato en estar disponible para el frontend. Para esto último consulta el último registro con el mismo endpoint que usa la aplicación y repite la consulta cada 100 ms hasta ver el número de registro enviado. El resultado es una medición aproximada de sensor a frontend que excluye el tramo inalámbrico entre el nodo y el hub, el cual se mide por separado en la serie de alcance y latencia de la sección 12.3.

El mismo script ejecuta las pruebas de rechazo: una medición sin el encabezado del token y otra con un token incorrecto, que deben responder 401, y una medición sin `sensor_id` y con un valor no numérico, que debe responder 400 con la lista de errores y sin registrar nada. Su salida se imprime como tablas listas para anexar. La ejecución contra el servidor desplegado se registra en el Anexo H.

Orden de ejecución: `node pruebas/medir_tiempos.js --url <dirección del servidor> --token <token del dispositivo> --email <correo> --clave <contraseña>`. Si el servidor exige sesión para las consultas del frontend, las credenciales permiten medir el tiempo hasta el frontend; sin ellas el script mide solo la respuesta del backend. Con el token global antiguo se agrega la opción `--vivienda casa-001`.

13. Evidencias

Se indica el estado de cada evidencia. Las marcadas como por anexar se toman del prototipo actual y se agregan a la entrega.

Evidencia	Ubicación	Estado
Código fuente del hub, del nodo C3 y del backend	Repositorio Git: https://github.com/HamedJason/ProyectoDeCarrera/tree/f72fce5133cfd960bd7d48f17254869be9f5a80e	Incluido. El enlace apunta exactamente al código descrito en este documento
Diagrama de conexión	Sección 4	Incluido, actualizado
Diagrama de flujo implementado	Sección 3	Incluido, actualizado
Estructura de la base de datos y su diagrama	Sección 7, diagrama 7.1 y <code>schema.sql</code>	Incluido, con diagrama entidad-relación
Repositorio Git con README y guía de despliegue	https://github.com/HamedJason/ProyectoDeCarrera	Incluido. README.md y PASOS.md en la carpeta poc-alarma
Backend desplegado	Anexo A, respuesta del endpoint de salud	Por anexar. Captura de GET /salud en el servidor desplegado
Serie de diez transmisiones	Sección 12.1 y Anexo H	Serie del 16 de septiembre incluida. Anexo H: repetición con el script

Evidencia	Ubicación	Estado
Respuestas del backend ante dato válido e incorrecto	Sección 12.4 y Anexo H, códigos 201, 400 y 401	Códigos 201 y 400 incluidos. Código 401 en el Anexo H
Fotografía del montaje con el nodo C3, los sensores y el hub	Anexo B	Por anexar, el montaje cambió
Monitor serie del hub recibiendo eventos por ESP-NOW	Anexo E	Por anexar
Capturas de la aplicación con los sensores y el historial	Anexo F	Por anexar
Captura de la notificación push recibida en el teléfono	Anexo G	Por anexar
Script de pruebas de transmisión, rechazo y tiempo hasta el frontend	Sección 12.6 y pruebas/medir_tiempos.js (rama main del repositorio)	Incluido en el repositorio, posterior al commit f72fce5. Resultado en el Anexo H
Video breve de funcionamiento	No requerido	El docente indicó que no será necesario para esta entrega

14. Cronograma de las semanas restantes

El cronograma abarca desde el 22 de septiembre hasta el 22 de noviembre. La tabla conserva su estructura original y añade el estado de cada semana. Varias tareas se adelantaron porque el frontend y el enlace inalámbrico avanzaron más rápido de lo previsto, y las pruebas formales se concentran en la semana 8.

Semana	Periodo	Actividad principal	Estado
1	22 al 28 de septiembre	Frontend mínimo conectado al backend. Sustituir el push button por el MC-38. Pruebas de pérdida de conexión y de token ausente	Frontend y sensor MC-38 hechos. Las dos pruebas formales siguen pendientes
2	29 de septiembre al 5 de octubre	Integrar el PIR AM312. Construir el primer nodo ESP32-C3 y migrar el enlace nodo a hub a ESP-NOW. Pruebas de alcance, pérdida y reintentos	PIR y nodo ESP-NOW funcionando. Faltan las pruebas de alcance y pérdida
3	6 al 12 de octubre	Integrar la ESP32-S3 CAM. Capturar evidencia fotográfica asociada a un evento y subirla por un canal separado del tráfico de sensores	Pendiente
4	13 al 19 de octubre	Ampliar el modelo de base de datos a viviendas, zonas, nodos, sensores, eventos y evidencia. Migrar los datos	Parcial. Ya existen las tablas de sensores, estado y acciones
5	20 al 26 de octubre	Armado y desarmado, retardos de entrada y salida, modo audible y silencioso. Sirena real con relevador	Armado, desarmado y modo silencioso hechos. Faltan los retardos y la sirena real
6	27 de octubre al 2 de noviembre	Migrar hub a backend a MQTT sobre TLS. Cola local y sincronización sin duplicados	Pendiente. La cola y el buffer existen sobre HTTP
7	3 al 9 de noviembre	Completar la aplicación: configuración, historial, evidencia y notificaciones push	Notificaciones push y configuración de sensores hechas. Falta la evidencia fotográfica
8	10 al 16 de noviembre	Pruebas cuantitativas de los indicadores del proyecto	Plan definido en la sección 12.3

Semana	Periodo	Actividad principal	Estado
9	17 al 22 de noviembre	Integrar resultados, documentación final, video y entrega del expediente	Pendiente

15. Presupuesto preliminar

El presupuesto conserva los tres bloques de la primera versión: el material disponible, el pedido realizado y lo que todavía falta por adquirir. El pedido ya fue recibido, por lo que el segundo bloque pasa a ser material disponible. Los costos corresponden a valores reales y no a estimaciones. No se registraron compras adicionales en esta etapa.

15.1 Material disponible antes del pedido

Componente	Cantidad	Costo unitario	Subtotal
ESP32 DevKit V1 (controlador principal)	1	\$180.00	\$180.00
Push button, protoboard y cables dupont	1	\$150.00	\$150.00
LED y resistencia de 220 ohm	1	\$15.00	\$15.00
Subtotal disponible			\$345.00

15.2 Pedido recibido

Componente	Cantidad	Costo unitario	Subtotal
ESP32-C3 SuperMini (nodos inalámbricos)	8	\$43.10	\$344.80
ESP32-S3 CAM con sensor OV5640	1	\$221.90	\$221.90
Sensores magnéticos MC-38 con cable, para puerta y ventana	10	\$8.04	\$80.43
Interruptores reed magnéticos MKA-0710	10	\$4.95	\$49.50
Sensores PIR AM312	4	\$22.39	\$89.56
Mini bocina de alarma 12 V CC, modelo PS-89	1	\$46.90	\$46.90
Módulo elevador y reductor TPS63020 a 3.3 V	5	\$16.78	\$83.90
Portapilas AAA con interruptor	10	\$10.79	\$107.90
Conectores JST PH 2.0 de 2 pines	10	\$8.09	\$80.90
Bloques de terminales de tornillo 5.08 mm de 2 polos	10	\$1.36	\$13.61
Tiras de headers 1x40 macho y hembra de 2.54 mm	10	\$4.79	\$47.90
Kit de resistencias de 1/4 W al 1 por ciento	600	\$0.13	\$79.90
Kit de capacitores electrolíticos de 12 valores	120	\$0.53	\$63.90
Capacitores cerámicos de 100 nF	100	\$0.10	\$10.04
Kit de botones táctiles de 6x6 mm	100	\$0.36	\$35.50
Micro interruptores de límite NO/NC de 5 A con palanca	10	\$3.19	\$31.90

Componente	Cantidad	Costo unitario	Subtotal
Placas adaptadoras de SMD a DIP, encapsulado SOT-23	5	\$6.58	\$32.90
Kit de soldadura eléctrica 10 en 1, 60 W	1	\$95.90	\$95.90
Tapete de silicona para estación de soldadura	1	\$47.90	\$47.90
Subtotal del pedido realizado			\$1565.24

Los precios unitarios corresponden al costo con descuento al momento de la compra. A los \$1565.24 pesos en artículos se agregaron \$184.14 pesos de envío de vendedores internacionales y se descontaron aproximadamente \$84.00 pesos en promociones, por lo que el pedido quedó en alrededor de \$1665.38 pesos.

15.3 Material por adquirir

La bocina opera a 12 V y no puede conectarse directamente a una salida del microcontrolador, que entrega 3.3 V y una corriente muy limitada. Se requiere una etapa de potencia y una fuente independiente, que deben conseguirse antes de la semana 5.

Componente	Cantidad	Costo unitario	Subtotal
Módulo relevador de 5 V o transistor MOSFET para la bocina	1	\$40.00	\$40.00
Fuente de alimentación de 12 V y 2 A para la bocina	1	\$150.00	\$150.00
Pilas AAA recargables para los nodos	8	\$35.00	\$280.00
Gabinete plástico para el controlador principal y la cámara	2	\$120.00	\$240.00
Rollo de filamento para impresión 3D (soportes y carcasas del prototipo)	1	\$500.00	\$500.00
Subtotal por adquirir			\$1210.00

15.4 Costos recurrentes

- Servidor privado virtual, ya contratado, sin costo adicional atribuible a este proyecto.
- Base de datos PostgreSQL alojada en el mismo servidor, sin costo adicional.
- Nombre de dominio mediante DNS dinámico gratuito, con la obligación de confirmarlo cada treinta días.
- Notificaciones push propias mediante Web Push, que no requieren un servicio de pago.
- Almacenamiento de evidencia fotográfica, entre 30 y 60 pesos mensuales según el volumen de imágenes, cuando se implemente la cámara.

El costo total del material asciende a alrededor de \$3120.24 pesos sin considerar envíos, de los cuales \$345.00 corresponden al material previo, \$1565.24 al pedido y \$1210.00 a lo que falta por adquirir, incluido un rollo de filamento de \$500.00. La cifra supera los \$2040 estimados en el Taller 4 porque el pedido reunió más piezas que la lista original, como se explica en la sección 2. El presupuesto lo financia el estudiante.

16. Matriz de riesgos

Los riesgos se ordenan por su efecto sobre el núcleo obligatorio. Se actualizaron con lo aprendido en esta etapa. Los marcados como nuevos no estaban en la primera versión.

Riesgo	Prob.	Impacto	Descripción	Acción preventiva o de contingencia
Token y tráfico del hub sin cifrar (nuevo)	Media	Alto	El hub envía por HTTP y el token del dispositivo viaja sin cifrar y está escrito en el firmware. Cualquiera que observe el tráfico podría copiarlo y enviar eventos falsos	Pasar el firmware a HTTPS, mover el token a un archivo fuera del repositorio y definir credenciales por dispositivo antes del prototipo alfa
Retraso de las notificaciones push (nuevo)	Media	Medio	Se observó que las notificaciones tardan más de lo esperado. En iPhone además dependen de la versión del sistema, de instalar la app en inicio y del ahorro de energía	Medir cada tramo con la serie de la sección 12.3. Mantener ntfy como canal alternativo opcional y documentar las condiciones para recibir el aviso
Canal de ESP-NOW y dirección del hub (nuevo)	Media	Medio	El hub cambia de canal si cambia el router, y el nodo guarda la dirección MAC del hub. Sustituir la placa del hub exige recargar los nodos	El nodo busca el canal de forma automática. Documentar el procedimiento de cambio de placa y probar la recuperación
Alcance insuficiente de ESP-NOW dentro de la vivienda	Media	Alto	Las paredes y la distancia pueden reducir el alcance por debajo de lo necesario. La ESP32-C3 SuperMini usa antena cerámica integrada	Medir el alcance con la serie de la sección 12.3 antes de armar más nodos. Si es insuficiente, reubicar el hub o incorporar un repetidor
Caducidad del nombre de dominio dinámico	Media	Alto	El proveedor elimina los nombres de cuentas gratuitas si no se confirman cada treinta días, y los nodos dejarían de localizar el servidor	Confirmar el nombre cada mes con un recordatorio fijo. Como respaldo, apuntar temporalmente a la dirección IP del servidor
Calidad insuficiente de la cámara	Baja	Medio	El sensor OV5640 puede dar imágenes poco útiles con poca luz o contraluz	Ejecutar el plan de pruebas de cámara en la semana 3 y ajustar exposición y resolución antes de considerar otro hardware
Falta de etapa de potencia para la bocina	Alta	Medio	La bocina es de 12 V y no se puede conectar directamente al microcontrolador. El relevador y la fuente no vinieron en el pedido	Conseguirlos antes de la semana 5. Mientras tanto la salida se valida con un LED, que comparte el mismo camino de control
Autonomía de batería menor a la esperada	Alta	Medio	El firmware actual mantiene la radio encendida todo el tiempo, por lo que con pilas duraría horas o pocos días. Es una estimación a partir de consumos típicos, sin medición propia	Alimentar los nodos por USB durante las pruebas. Medir el consumo real y, si se requiere batería, diseñar el reposo profundo con despertar por entrada
Tiempo insuficiente para las funciones experimentales	Alta	Bajo	La detección de personas podría no alcanzar a implementarse	Mantenerla como función experimental. El cronograma prioriza el núcleo obligatorio
Inestabilidad del Internet residencial	Media	Medio	Los cortes de la conexión doméstica afectan el registro remoto	El hub conserva los eventos en un buffer y los sincroniza. Documentar cada corte observado
Pérdida de código o de configuración	Baja	Alto	Un fallo del equipo o un cambio mal aplicado podría comprometer el avance	Mantener el repositorio actualizado con subidas al menos semanales. Las credenciales se guardan fuera del repositorio

17. Actualización del alcance

El desarrollo de esta etapa no modifica el alcance aprobado, pero sí hay cuatro precisiones que conviene dejar registradas.

- **Enlace inalámbrico adelantado.** ESP-NOW entre el nodo y el hub, programado para la semana 2, ya funciona en banco. Su confiabilidad todavía debe validarse con las pruebas de alcance y pérdida, y no debe afirmarse que sea perfecta sin ellas.
- **Aplicación web instalable en lugar de aplicación nativa.** La interfaz del usuario es una aplicación web que se instala en la pantalla de inicio de Android y de iOS. Se eligió porque una sola base de código cubre ambas plataformas y permite notificaciones push sin pasar por las tiendas. El alcance habla de una aplicación móvil Android e iOS. Esta decisión es propia y se propone confirmarla con el docente antes de consolidarla.
- **Notificaciones push propias.** En lugar de un servicio externo pendiente de seleccionar, se implementó Web Push directamente en el backend, lo que elimina una dependencia de terceros.
- **Funciones adelantadas.** El armado y desarmado, el modo silencioso, la trazabilidad de acciones y las notificaciones se adelantaron respecto al cronograma original. Los retardos de entrada y salida siguen pendientes y deben cubrirse en la semana 5.

La viabilidad de la arquitectura quedó confirmada en banco para la cadena completa, desde el sensor hasta la pantalla y la notificación. Lo que todavía no está confirmado son los indicadores cuantitativos, que requieren las series de la sección 12.3. Se mantiene también la dependencia del nombre de dominio dinámico, incorporada en la matriz de riesgos.

18. Matriz de cumplimiento de los criterios de la actividad

La tabla relaciona cada criterio de la actividad con su estado y con el lugar donde se comprueba. El estado Parcial indica que el elemento existe pero falta anexar una evidencia o ejecutar una medición, y se indica cuál. El anexo correspondiente se agrega a la entrega cuando se obtiene.

Criterio de la actividad	Estado	Dónde está la evidencia
Leer un sensor o entrada física real	Cumple	Contactos MC-38 y sensor AM312 sobre un nodo ESP32-C3. Secciones 4, 5.1 y 11
Realizar un procesamiento básico del dato	Cumple	Filtrado de rebote de 50 ms, estabilización y bloqueo del PIR, deduplicación y lógica de armado en el hub. Secciones 5.1 y 5.2
Conectarse a Internet	Cumple	El hub ESP32 se conecta por Wi-Fi con reconexión no bloqueante. Secciones 5.2 y 12.1
Enviar un tipo de dato al backend	Cumple	POST /mediciones con el formato del hub. Secciones 6 y 12.1
Identificar cada medición por fecha, hora o número de registro	Cumple	Número de registro consecutivo generado por el hub y fecha y hora asignadas por el backend. Sección 5.3
Backend desplegado temporalmente en la nube	Cumple	Servidor privado virtual con contenedores, accesible desde Internet. Sección 6. Falta anexar la captura del endpoint de salud (Anexo A)
Recibir un tipo de dato del sistema embebido	Cumple	POST /mediciones recibe los eventos del hub. Secciones 6 y 12.1

Criterio de la actividad	Estado	Dónde está la evidencia
Validar los campos indispensables	Cumple	Respuestas 400 con la lista de errores y 401 sin token. Sección 12.4 y script de la 12.6
Registrar la información en la base de datos	Cumple	Cada medición válida se guarda en la tabla mediciones con identificador y fecha. Secciones 7 y 12.1
Permitir que el frontend consulte la información	Cumple	GET /mediciones y GET /mediciones/ultima, usados por la aplicación. Secciones 6 y 8
Una tabla o colección, identificador, variable y valor, unidad, fecha y hora	Cumple	Tabla mediciones. Sección 7 y comprobación en la 7.2
Mínimo diez registros reales	Cumple	Identificadores 6 al 15 de la serie de la sección 12.1, más el uso posterior del prototipo
Una consulta para recuperar la información	Cumple	Consulta de las últimas 50 mediciones. Sección 7
Diez transmisiones consecutivas	Cumple	Sección 12.1, con el código HTTP y la latencia de cada una
Registro de transmisiones exitosas y fallidas	Cumple	Tabla de indicadores de la sección 12.1: 10 exitosas y 0 fallidas
Medición aproximada del tiempo sensor a frontend	Parcial	Medido hasta el backend: 350.6 ms en promedio (12.1). El tiempo hasta el frontend se obtiene con el script de la sección 12.6 y se anexa en el Anexo H
Prueba con un dato incorrecto o una pérdida temporal de conexión	Cumple	Dato incorrecto con respuesta 400 verificada en el servidor desplegado (12.4). Pérdida de conexión observada con el buffer en 12.2
Registro breve de problemas y correcciones	Cumple	Once problemas con su causa y su corrección. Sección 12.5
Código fuente inicial	Cumple	Repositorio Git, commit f72fce5 (enlace en la portada y en la sección 13)
Diagrama de conexión y flujo implementado	Cumple	Figuras 1 y 2, secciones 3 y 4
Video breve de funcionamiento	No requerido	El docente indicó que no era necesario para esta entrega. Se puede grabar si se solicita
Evidencia del backend desplegado	Parcial	Servicio funcionando (sección 11). Falta la captura de GET /salud en el Anexo A
Estructura inicial de la base de datos	Cumple	Sección 7, diagrama de la sección 7.1 y schema.sql
Frontend mostrando datos reales	Parcial	Funcionalidad descrita en la sección 8. Faltan las capturas de la aplicación con datos reales en el Anexo F
Repositorio Git con README básico	Cumple	Enlace en la portada y en la sección 13. README.md y PASOS.md
Cronograma de las semanas restantes	Cumple	Sección 14
Presupuesto preliminar	Cumple	Sección 15

Criterio de la actividad	Estado	Dónde está la evidencia
Matriz de mínimo cinco riesgos con acciones preventivas o de contingencia	Cumple	Sección 16